string in place of the *0xD8* bytes. Conversely, under UTF-16 encoding, the sequence *0xD8 0x00* is a single invalid sequence and is substituted with the UTF-16 encoding of U+FFFD, which is (conveniently) *0xFF 0xFD*.

`mb_convert_kana()`

Finally, we have the Japanese language-specific `mb_convert_kana()`. It might seem odd that there isn't also, say, `mb_convert_hebrew()`, `mb_convert_sanskrit()`, or `mb_convert_cuneiform()`. I suspect that is because the authors of the `mbstring` extension are named Tsukada Takuya and Rui Hirokawa (https://github.com/php/php-src/blob/master/ext/mbstring/CREDITS), and that's fair enough, I suppose. Domo arigato gozaimashita for your hard work, both of you. This function takes an input string, a "mode" parameter, and an optional encoding type (otherwise the `internal_encoding` settings are used).

This function does have some utility, in that it converts between fullwidth and halfwidth characters. Should you need to mix and match East Asian and Latin symbols, it is possible to use this function:

```
print "関数があります。" . PHP_EOL;
print "abcdefg" . PHP_EOL;
print mb_convert_kana("abcdefg", 'A', 'UTF-8');
```

```
Output:
関数があります。
abcdefg
ａｂｃｄｅｆｇ
```

Cryptography

The desire for secrecy in communications is as old as, well, human communication. Transforming written language into a form that is indecipherable during transmission or storage and requiring special knowledge at the receiving end to decipher is a task that drove the development of modern computers as we know them. This section will not be an exhaustive analysis of cryptography (you might be glad to hear!) but will hopefully equip you with a basic understanding of the tools and some common use cases. Let's see what all the fuss is about.